

1:List the Employee Name, their Department Name, and the Location Name for all employees.

```
SELECT e.employee_name, d.dept_name, d.location_id FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id;
```

2:Retrieve the Project Name, Client Name, and Total Hours Worked, but only for projects managed by the 'Consulting' department.

```
SELECT p.project_name, p.client_name, SUM(a.hours_worked) AS total_hours
FROM Projects p
JOIN Departments d ON p.dept_id = d.dept_id
JOIN Assignments a ON p.project_id = a.project_id
WHERE d.dept_name = 'Consulting' GROUP BY p.project_name, p.client_name;
```

3:Find employees who have worked on a project with Client Name 'GlobalTech Solutions'. List Employee Name and Project Name.

```
SELECT e.employee_name, p.project_name FROM Employees e
JOIN Assignments a ON e.employee_id = a.employee_id
JOIN Projects p ON a.project_id = p.project_id WHERE p.client_name =
'GlobalTech Solutions';
```

4: List all Departments that have no Employees currently assigned to them.

```
SELECT d.dept_name FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id WHERE e.employee_id IS
NULL;
```

5:Show the Manager Name, the Department Name they manage, and the total number of direct reports.

```
SELECT m.employee_name AS manager_name, d.dept_name,
COUNT(e.employee_id) AS direct_reports FROM Employees e
JOIN Employees m ON e.manager_id = m.employee_id
JOIN Departments d ON m.dept_id = d.dept_id
GROUP BY m.employee_name, d.dept_name;
```

6:Calculate the Average Hours Worked per Employee across all projects. List Employee Name and average hours.

```
SELECT e.employee_name, AVG(a.hours_worked) AS avg_hours FROM
Employees e
JOIN Assignments a ON e.employee_id = a.employee_id GROUP BY
e.employee_name;
```

7:Find Department Name(s) with an average salary above the company-wide average salary.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM Employees);
```

8:Identify Client Name(s) whose projects have total combined hours exceeding 500.

```
SELECT p.client_name, SUM(a.hours_worked) AS total_hours FROM Projects p
JOIN Assignments a ON p.project_id = a.project_id
GROUP BY p.client_name HAVING SUM(a.hours_worked) > 500;
```

12:Retrieve Department Name(s) that have employees assigned to more than one project.

```
SELECT DISTINCT d.dept_name FROM Departments d
JOIN Employees e ON d.dept_id = e.dept_id
JOIN Assignments a ON e.employee_id = a.employee_id GROUP BY
d.dept_name, e.employee_id HAVING COUNT(DISTINCT a.project_id) > 1;
```

13:List Employee Name and Project Name for any employee who has worked less than 10 hours on a project.

```
SELECT e.employee_name, p.project_name, a.hours_worked FROM Employees
e
JOIN Assignments a ON e.employee_id = a.employee_id
JOIN Projects p ON a.project_id = p.project_id WHERE a.hours_worked < 10;
```

14:Find Project Name(s) that have no assigned employees.

```
SELECT p.project_name FROM Projects p
LEFT JOIN Assignments a ON p.project_id = a.project_id WHERE a.employee_id
IS NULL;
```

15:Select Employee Name(s) whose salary is greater than the maximum salary in their own department.

```
SELECT e.employee_name, e.salary FROM Employees e
WHERE e.salary > (
    SELECT MAX(e2.salary) FROM Employees e2
    WHERE e2.dept_id = e.dept_id AND e2.employee_id != e.employee_id
);
```

16: Find the Employee Name and their oldest project start date.

```
SELECT e.employee_name, MIN(a.start_date) AS oldest_start_date FROM
Employees e
JOIN Assignments a ON e.employee_id = a.employee_id GROUP BY
e.employee_name;
```

17:Using DATE_FORMAT, list all projects that were active in the year 2023.

```
SELECT project_name, DATE_FORMAT(start_date, '%Y-%m-%d') AS
formatted_date FROM Projects
WHERE DATE_FORMAT(start_date, '%Y') = '2023';
```

18: Using IFNULL, list all Employee Names; display "CEO" if an employee has no manager.

```
SELECT employee_name, IFNULL(CAST(manager_id AS CHAR), 'CEO') AS
manager FROM Employees;
```

19:Find the top 3 Departments based on total hours worked across all projects.

```
SELECT d.dept_name, SUM(a.hours_worked) AS total_hours FROM
Departments d
JOIN Employees e ON d.dept_id = e.dept_id
JOIN Assignments a ON e.employee_id = a.employee_id
GROUP BY d.dept_name ORDER BY total_hours DESC LIMIT 3;
```

20:Display Project Name and a status ("High Activity" if total hours > 300, "Normal" otherwise) using CASE.

```
SELECT p.project_name,
       CASE WHEN SUM(a.hours_worked) > 300 THEN 'High Activity' ELSE
'Normal' END AS status
FROM Projects p
JOIN Assignments a ON p.project_id = a.project_id
GROUP BY p.project_name;
```

21:List Employee Name and Manager Name, even if the employee has no manager or vice versa.

```
SELECT e.employee_name, m.employee_name AS manager_name
FROM Employees e
FULL OUTER JOIN Employees m ON e.manager_id = m.employee_id;
```

